Graph Traversal & Search Algorithms

A Comprehensive Analysis of BFS, DFS, Best-First, A*, and AO*

Foundations of Graph Search

Uninformed Search

Also known as blind search, these algorithms operate without any domain-specific knowledge about the target's location.

- Rely solely on the structure of the graph.
- Explore nodes systematically based on entry order.
- **Examples:** Breadth-First Search (BFS), Depth-First Search (DFS).

Informed (Heuristic) Search

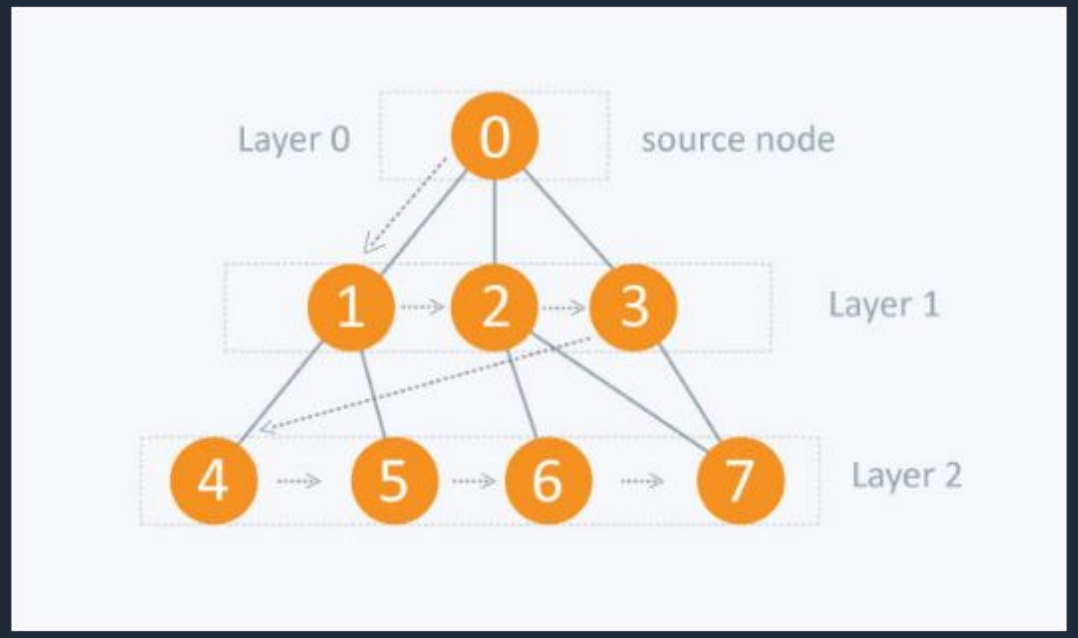
Utilize a heuristic function to estimate the cost or distance to the goal, significantly optimizing the search path.

- Make educated guesses on which path is best.
- Evaluate nodes using specific mathematical functions.
- **Examples:** Greedy Best-First Search, A*, AO*.

Breadth-First Search (BFS)

BFS is an uninformed search algorithm that explores the graph level by level, ensuring that all nodes at the present depth are examined before moving deeper.

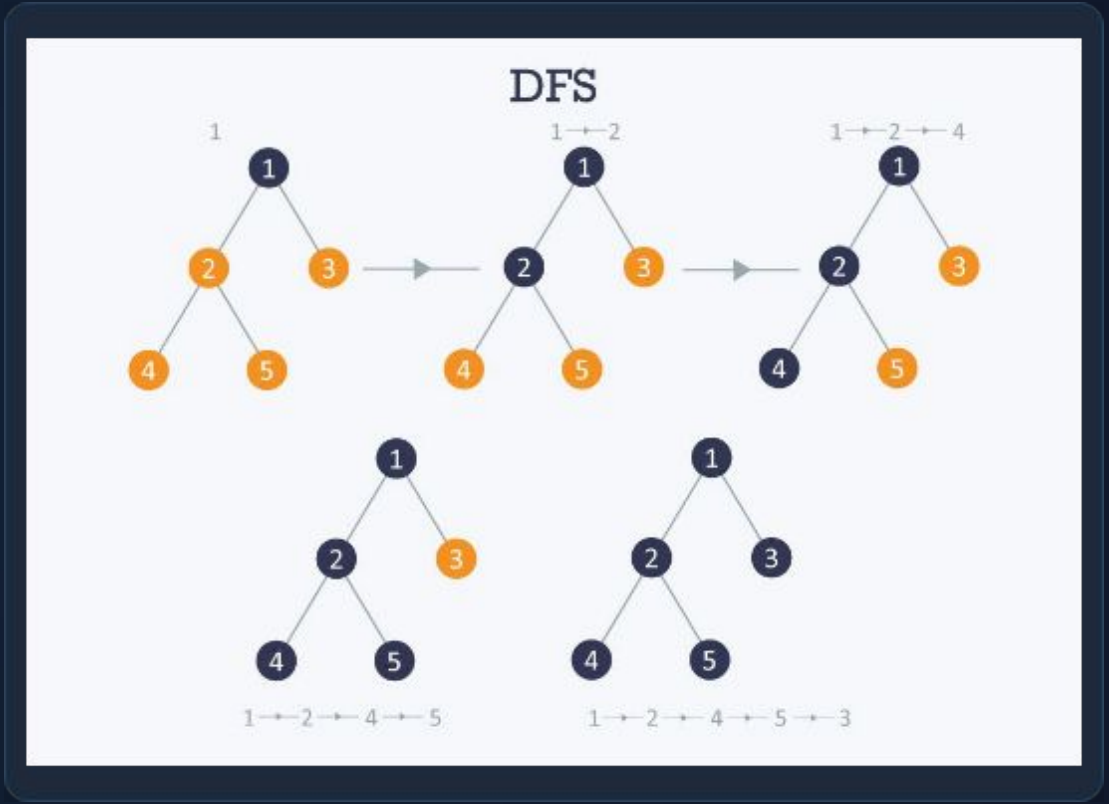
- **Data Structure:** Utilizes a Queue (FIFO - First In, First Out).
- **Target Finding:** Guarantees the shortest path in unweighted graphs.
- **Mechanism:** Systematically visits all direct neighbors of a node before moving to the neighbors' neighbors.
- **Drawback:** Highly memory-intensive as it must store all nodes of a level.



Depth-First Search (DFS)

DFS is an uninformed search algorithm that explores as far as possible along each branch before backtracking.

- **Data Structure:** Utilizes a Stack (LIFO - Last In, First Out) or recursion.
- **Target Finding:** May not find the shortest path, but excels at reaching deep targets quickly.
- **Mechanism:** Dives straight down a path until a dead end is reached, then backtracks to the last branching point.
- **Advantages:** Significantly lower memory footprint compared to BFS. Used for topological sorting and maze generation.



Mathematics of Uninformed Search

Complexity Constraints

Evaluating graph algorithms requires understanding their computational boundaries in terms of Time and Space (Memory).

- **Vertices (V):** The total number of nodes in the graph.
- **Edges (E):** The total number of connections between the nodes.
- Both BFS and DFS must, in the worst case, traverse every vertex and every edge to find a target or conclude it does not exist.

Time Complexity (BFS & DFS)

$$O(|V| + |E|)$$

Space Complexity (Worst Case)

$$O(|V|)$$

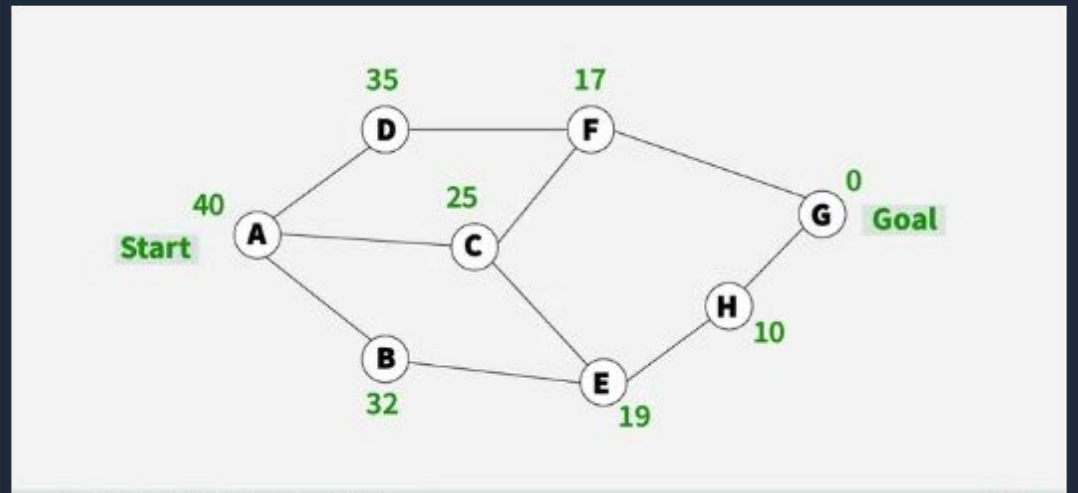
Heuristic Search

Moving beyond blind exploration by utilizing cost estimations and problem-specific knowledge to radically accelerate pathfinding.

Greedy Best-First Search

An informed search that expands the node that appears to be closest to the target, prioritizing immediate, localized gains.

- **Data Structure:** Utilizes a Priority Queue.
- **Heuristic Function:** Evaluates nodes using $f(n) = h(n)$, where $h(n)$ is the estimated cost to the goal.
- **Mechanism:** Strictly follows the heuristic gradient, disregarding the cost it took to get to the current node.
- **Drawback:** Fast, but highly susceptible to false leads and obstacles; rarely optimal.



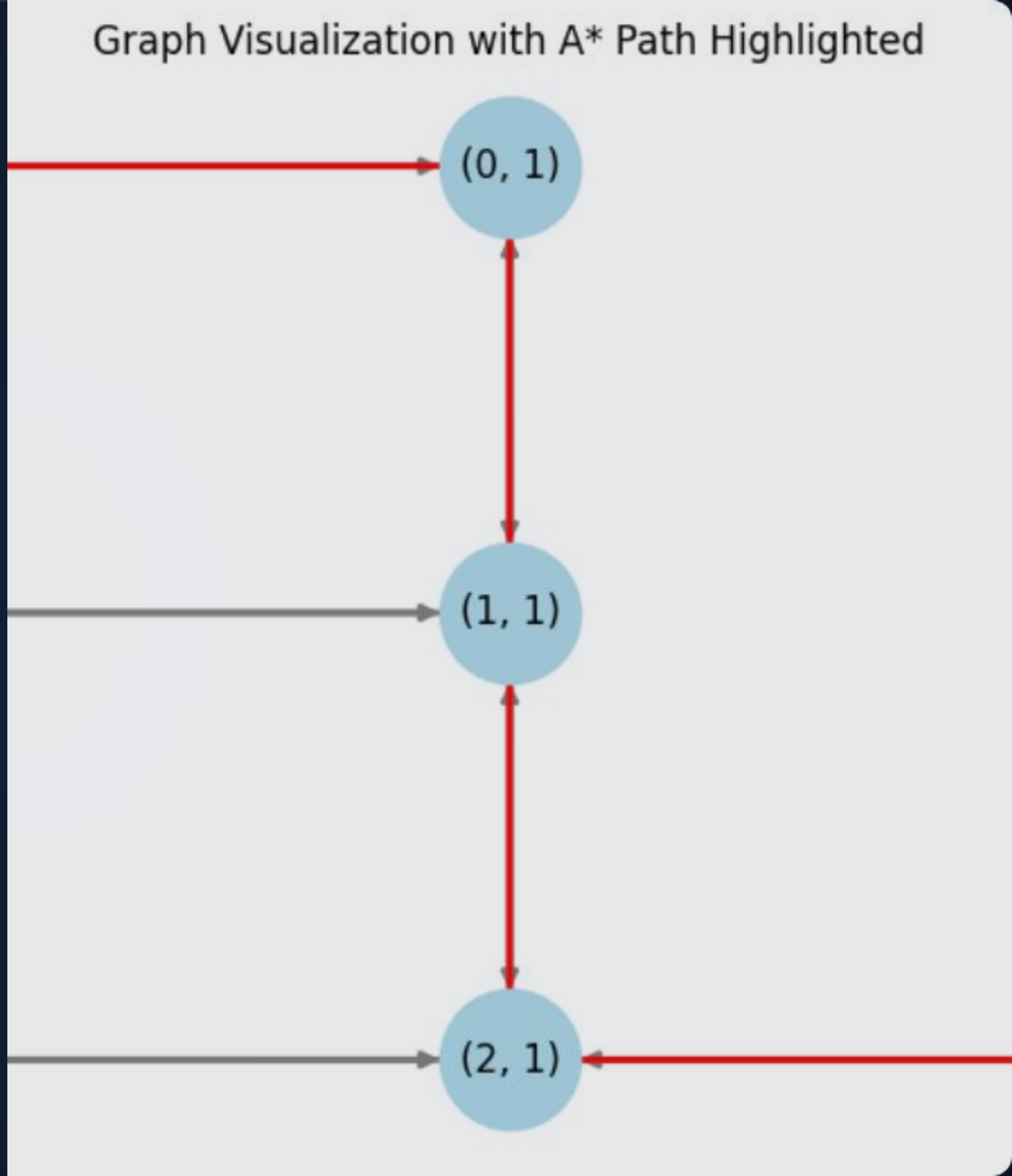
Best-First Search algorithm

A* Search Algorithm

The Standard for Optimal Pathfinding

A* brilliantly balances the actual distance traveled so far with the estimated distance to the target.

By preventing the algorithm from wandering aimlessly (like BFS) while also preventing it from being tricked by obstacles (like Greedy Search), A* guarantees the absolute shortest path, provided the heuristic is admissible (never overestimates).



The Mathematics of A*

The Evaluation Function

A* determines which node to expand next by calculating the lowest total estimated cost, denoted as f(n).

- **g(n):** The exact, known cost from the starting node to the current node 'n'.
- **h(n):** The heuristic estimated cost from node 'n' to the final goal.

For the algorithm to remain optimal, h(n) must be *admissible*—it must never overestimate the true cost to reach the target.

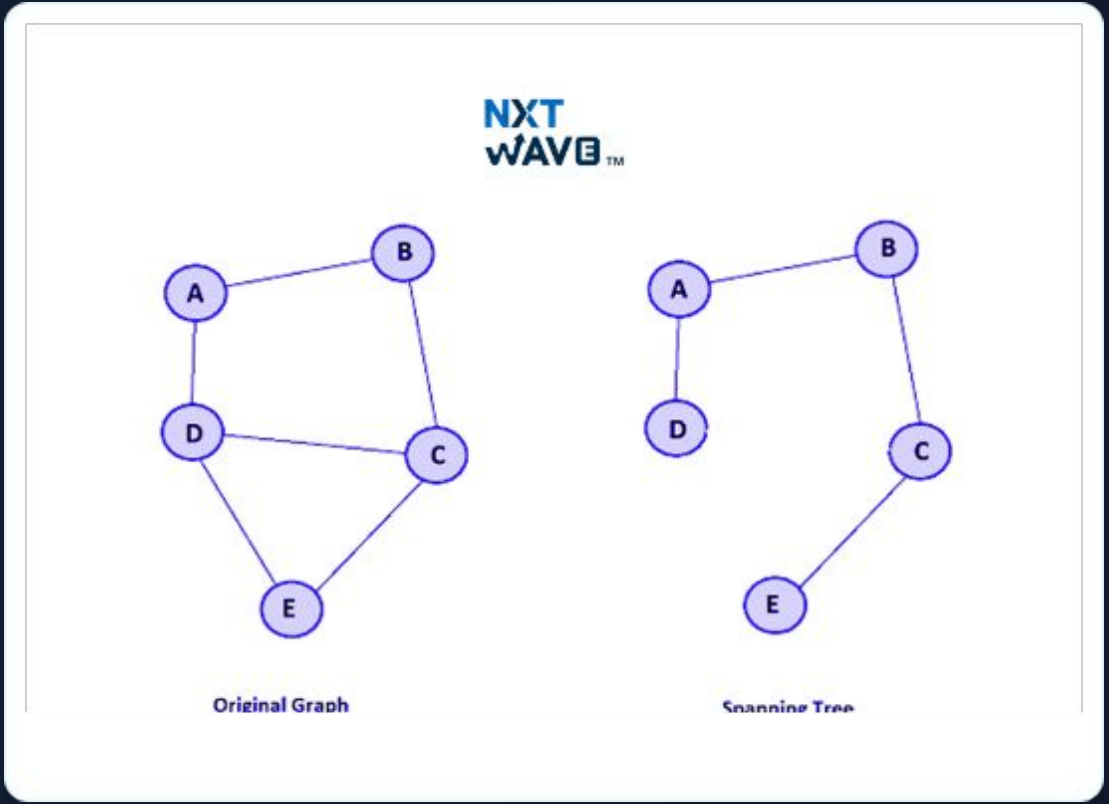
A* Core Equation

$$f(n) = g(n) + h(n)$$

AO* Algorithm & AND-OR Graphs

While A* finds a single path, AO* is designed for *Problem Decomposition*. It solves complex problems by breaking them into smaller sub-problems.

- **AND Nodes:** Represent a problem that has been decomposed into smaller parts. *All* child paths must be solved to resolve the node.
- **OR Nodes:** Represent alternative strategies. Only *one* child path needs to be solved to resolve the node.
- **Mechanism:** Explores the graph top-down, but propagates costs bottom-up to continuously revise the best strategy.



Cost Propagation in AO*

Calculating Strategy Costs

The cost of solving a problem in an AND-OR graph depends on the type of node being evaluated.

Costs are calculated recursively from the leaf nodes back up to the root to determine the most efficient problem-solving strategy.

- **OR Nodes:** Pick the cheapest available option.
- **AND Nodes:** Must pay the cost of every required sub-problem.

For an OR Node

$$\text{Cost}(n) = \min_{i} \left(\text{Cost}(n_i) + c_i \right)$$

For an AND Node

$$\text{Cost}(n) = \sum_{i=1}^{k} \left(\text{Cost}(n_i) + c_i \right)$$

Algorithm Comparison Matrix

Algorithm	Data Structure	Optimal?	Heuristic Usage	Primary Use Case
BFS	Queue (FIFO)	Yes (Unweighted)	None	Shortest path in unweighted networks
DFS	Stack (LIFO)	No	None	Topological sorts, maze solving
Best-First	Priority Queue	No	h(n) Only	Fast estimation, greedy approaches
A*	Priority Queue	Yes (Admissible h)	g(n) + h(n)	GPS routing, AI pathfinding in games
AO*	Tree / Graph	Yes	g(n) + h(n) on trees	Problem decomposition, expert systems